

CS 473 (Spring 2025)
Homework 6 (due Mar 27 Thu 10am)

Instructions: As in previous homeworks.

Problem 6.1: The original version of the smallest enclosing circle problem is very sensitive to so-called “outliers”: if a single point is corrupted, the optimal circle could change drastically. This motivates the following extension of the problem:

Given a set S of n points in 2D and an integer k , find the smallest circle that encloses at least $n - k$ points (i.e., we allow at most k points to be outside the circle).

In this question, you will design an efficient Monte Carlo algorithm to solve this new problem. You may assume that there are no degeneracies, i.e., no 4 points lie on a common circle.

- (a) (10 pts) Describe a naive algorithm that runs in $O(n^4)$ time.
- (b) (35 pts) Let C^* denote the optimal circle, let B^* denote the set of the (at most) 3 points on the boundary of C^* , and let Q^* denote the set of at most k points outside the circle. Form a random subset $R \subseteq S$ as follows: for each point $p \in S$, put p in R independently with probability $1/k$. (Consequently, $\mathbb{E}[|R|] = n/k$.)
Prove that the probability that $B^* \subseteq R \subseteq S - Q^*$ is at least $\Omega(1/k^3)$.
- (c) (40 pts) Describe a Monte Carlo algorithm that runs in $O(n)$ worst-case time and is correct with probability $\Omega(1/k^3)$. (Yes, this probability converges to 0, i.e., your algorithm is supposed to be wrong most of the time!)
[Hint: you may use the smallest enclosing circle algorithm from class as a subroutine; obviously, (b) is meant to help...]
- (d) (15 pts) Now, describe an efficient Monte Carlo algorithm that has error probability at most 0.01. Analyze the worst-case running time as a function of n and k . When k is small, it should be much faster than the algorithm in (a).

Solution:

(a) Brute force algorithm:

Algorithm 1

```

1: Initialize  $C^*$  as a circle with infinity radius.
2: for each triplet of points  $(p_i, p_j, p_k)$  and pair of points  $(p_i, p_j)$  do
3:   Construct the unique circle  $C$  that passes through all three or two points.
4:   Count the number of points inside or on the boundary of  $C$ .
5:   if at least  $n - k$  points are enclosed and the radius is smaller than  $C^*$  then
6:      $C^* = C$ .
7: return  $C^*$ 

```

Complexity Analysis:

There are $\binom{n}{3} + \binom{n}{2} = O(n^3)$ triplets and pairs, so the number of candidate circles is at most $O(n^3)$. Checking each circle takes $O(n)$ time to count the enclosed points. Hence, the total runtime is $O(n^3) \times O(n) = O(n^4)$.

(b)

$$B^* \subseteq R \subseteq S - Q^* \Leftrightarrow B^* \subseteq R \text{ and } Q^* \cap R = \emptyset$$

Define two independent events:

Event A : All points in B^* are included in R .Event B : No points from Q^* are included in R .Since each point is independently selected with probability $1/k$:

$$\Pr(A) = (1/k)^3 = 1/k^3.$$

Similarly, the probability that none of the at most k outlier points are selected is:

$$\Pr(B) = (1 - 1/k)^k.$$

Using the standard bound $(1 - 1/k)^k < 1/e$, $\Pr(B)$ remains a constant $\Theta(1)$.

Thus, the desired probability is:

$$\Pr(B^* \subseteq R \text{ and } Q^* \cap R = \emptyset) = \Pr(A) \cdot \Pr(B) = \frac{1}{k^3} \cdot \Theta(1) = \Omega(1/k^3).$$

(When $k = 1$, we can sample each p with probability $1/2$ so the probability still satisfies $\Omega(1/k^3)$)

(c) From part (b):

$$\Pr(B^* \subseteq R \text{ and } Q^* \cap R = \emptyset) = \Omega(1/k^3).$$

When this event occurs, the subset R contains all necessary boundary points of C^* and excludes all outliers. Consequently, the smallest enclosing circle computed from R must be C^* itself, which covers at least $n - k$ points. Thus, the algorithm produces the correct result with probability at least $\Omega(1/k^3)$.

Complexity Analysis:

Iterating over S and independently deciding whether to include each point in R takes $O(n)$ time. Given that the expected size of R is $O(n/k)$, the smallest enclosing circle runs in $O(n)$ worst-case time.

Algorithm 2

- 1: Construct a random subset $R \subseteq S$ by including each point $p \in S$ independently with probability $1/k$.
 - 2: Use Welzl's algorithm to compute the smallest enclosing circle C_R for the subset R .
 - 3: **return** C_R
-

- (d) Use the Monte Carlo algorithm from part (c) as a subroutine that, in one iteration, randomly construct R and compute C_R . Scan all points in S to count how many lie inside or on the boundary of C_R . If at least $n - k$ points are contained, output C_R as the solution; otherwise, declare this iteration a failure.

Let p be the probability that one iteration returns the correct circle. From part (b),

$$p = \Pr(B^* \subseteq R \text{ and } R \subseteq S - Q^*) = \Omega(1/k^3).$$

In order to reduce the error probability to at most 0.01, Repeat the above procedure T times independently so that:

$$(1 - p)^T \leq 0.01,$$

we obtain,

$$T \geq \left\lceil \frac{\ln 0.01}{\ln(1 - p)} \right\rceil.$$

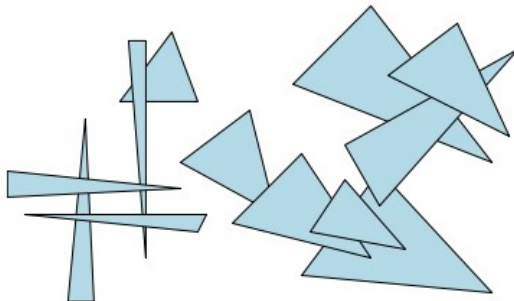
From part (b), $p < 1/e$, thus $\ln(1 - p) > -2p$, it suffices to take

$$T = O(k^3).$$

Complexity Analysis:

Each iteration runs in $O(n)$ worst-case time. Repeating the procedure $T = O(k^3)$ times gives an overall worst-case running time of $O(nk^3)$, which is much faster when k is small.

Problem 6.2: We are given n triangles $T_1, \dots, T_n$ in 2D. We want to compute the area of their union, i.e., $A = \text{area}(T_1 \cup \dots \cup T_n)$. (Note that the triangles may overlap, so A may not necessarily be equal to $\text{area}(T_1) + \dots + \text{area}(T_n)$.)



In this problem, we consider a randomized Monte Carlo algorithm to compute a good approximation of A . Let r be a parameter. Here is the pseudocode:

1. for $i = 1$ to n do
2. independently pick r random points $p_{i,1}, \dots, p_{i,r}$ from T_i
3. return $X = \sum_{i=1}^n |\{p_{i,1}, \dots, p_{i,r}\} \setminus (T_1 \cup \dots \cup T_{i-1})| \cdot \frac{\text{area}(T_i)}{r}$

(a) (30 pts) Prove that $\mathbb{E}[X] = A$.

[Hint: what is $\sum_{i=1}^n \text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1}))$?]

(b) (30 pts) Prove that $\text{Var}[X] \leq A^2/r$. ($\text{Var}[X]$ denotes the variance of X .)

(c) (40 pts) By choosing r appropriately, show that the algorithm runs in $O(n^2)$ time and outputs a value X between $0.99A$ and $1.01A$ with probability at least 0.99 .

[Note: you may assume that we can generate a random point in a triangle in $O(1)$ time, we can compute the area of a triangle in $O(1)$ time, and we can test whether a point is inside a triangle in $O(1)$ time. The above algorithm actually works for other types of objects besides triangles, and in dimension higher than 2.]

Solution:

(a) For each triangle T_i , define

$$X_i := |\{p_{i,1}, \dots, p_{i,r}\} \setminus (T_1 \cup \dots \cup T_{i-1})| \cdot \frac{\text{area}(T_i)}{r}.$$

$$a_i = \text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1})),$$

Consider each random point $p_{i,k}$ out of r points selected from T_i . Since the points are sampled uniformly, the probability that $p_{i,k}$ falls within the region $T_i \setminus (T_1 \cup \dots \cup T_{i-1})$ is

$$\Pr[p_{i,k} \in T_i \setminus (T_1 \cup \dots \cup T_{i-1})] = \frac{a_i}{\text{area}(T_i)}.$$

Each such point contributes $\frac{\text{area}(T_i)}{r}$, so its expected contribution is

$$\frac{\text{area}(T_i)}{r} \cdot \frac{a_i}{\text{area}(T_i)} = \frac{a_i}{r}.$$

Since there are r points sampled from T_i , the total expected contribution from T_i is

$$\mathbb{E}[X_i] = r \cdot \frac{a_i}{r} = a_i.$$

By linearity of expectation, sum over all X_i :

$$\mathbb{E}[X] = \sum_{i=1}^n \mathbb{E}[X_i] = \sum_{i=1}^n a_i.$$

The regions $T_i \setminus (T_1 \cup \dots \cup T_{i-1})$ are disjoint and collectively form $T_1 \cup \dots \cup T_n$, by definition,

$$\sum_{i=1}^n a_i = \text{area}(T_1 \cup \dots \cup T_n) = A.$$

(b) Define

$$C_i = |\{p_{i,1}, \dots, p_{i,r}\} \setminus (T_1 \cup \dots \cup T_{i-1})|$$

$$a_i = \text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1})),$$

we have,

$$C_i \sim \text{Bin}\left(r, \frac{a_i}{\text{area}(T_i)}\right),$$

which is the probability that a random point in T_i is not covered by previous triangles.

$$\text{Var}[C_i] = r \cdot p_i \cdot (1 - p_i).$$

Thus,

$$\text{Var}[X_i] = \left(\frac{\text{area}(T_i)}{r}\right)^2 \cdot \text{Var}[C_i] = \frac{\text{area}(T_i)^2}{r^2} \cdot r p_i (1 - p_i) = \frac{a_i \text{area}(T_i)}{r} (1 - p_i).$$

Since $1 - p_i \leq 1$, we have:

$$\text{Var}[X_i] \leq \frac{a_i \text{area}(T_i)}{r}.$$

Since X_i are independent, the total variance is the sum of variances of X_i :

$$\text{Var}[X] = \sum_{i=1}^n \text{Var}[X_i] \leq \sum_{i=1}^n \frac{a_i \text{area}(T_i)}{r}.$$

Notice $\text{area}(T_i) \leq A$ for all i (since each triangle is part of the union). Thus,

$$\sum_{i=1}^n a_i \text{area}(T_i) \leq \sum_{i=1}^n a_i A = A \sum_{i=1}^n a_i = A \cdot A = A^2.$$

Substituting this bound:

$$\text{Var}[X] \leq \frac{A^2}{r}.$$

- (c) For each triangle T_i , generating r random points takes $O(r)$ time. For each point $p_{i,j}$, checking membership in $T_1 \cup \dots \cup T_{i-1}$ requires $O(i)$ operations (testing against $i - 1$ triangles, each in $O(1)$ time). Total time for all checks:

$$\sum_{i=1}^n r \cdot O(i) = O(rn^2).$$

Combining sampling and checks:

$$O(nr) + O(rn^2) = O(rn^2).$$

Choosing r as a constant gives $O(n^2)$ time.

By Chebyshev's Inequality, for $\epsilon = 0.01A$:

$$\Pr[|X - A| \geq 0.01A] \leq \frac{\text{Var}[X]}{(0.01A)^2} \leq \frac{A^2/r}{0.0001A^2} = \frac{10000}{r}.$$

To ensure $\frac{10000}{r} \leq 0.01$, set $r \geq 10^6$.

Problem 6.3: We are given a (unweighted) bipartite graph G with n vertices and m edges. Suppose we have already computed a maximum matching M in G . Now suppose we delete a vertex v from G and all its incident edges. Let G' be the new graph (with $n - 1$ vertices). Describe how to efficiently compute a new maximum matching M' in G' . (Your algorithm should be faster than re-running a matching algorithm from scratch. Remember to prove correctness of your algorithm.)

Solution:

Idea: According to Berge's theorem, if there is no augmenting path in the current bipartite graph, then the current matching has already reached the maximum matching size.

Algorithm 3

Require: Bipartite graph $G = (X \cup Y, E)$, a maximum matching M in G , and a vertex $v \in V(G)$ to delete.

Ensure: A maximum matching M' in the new graph $G' = G \setminus \{v\}$.

```

1: procedure UPDATEMATCHING( $G, M, v$ )
2:   Remove vertex  $v$  and all incident edges from  $G$  to form  $G'$ 
3:   if  $v$  is not matched in  $M$  then
4:     return  $M$  ▷ Matching remains valid
5:   else
6:      $u \leftarrow$  the vertex matched with  $v$  in  $M$ 
7:      $M' \leftarrow M \setminus \{(u, v)\}$ 
8:      $P \leftarrow$  augmenting path of  $(G', M')$  from  $u$  using BFS/DFS
9:     if  $P$  is found then
10:      updating  $M'$  by flipping the edges along the augmenting path  $P$ 
11:      return  $M'$ 
12:     else
13:      return  $M'$  ▷ Maximum matching decreases by one

```

Correctness and Complexity:

If the deleted vertex v is unmatched, the matching M remains maximum in G' .

If v is matched, removing it leaves vertex u unmatched. According to Berge's theorem, if an augmenting path is found starting from u , flipping the edges restores the matching size; otherwise, M' has already reached the maximum matching size, which means the maximum matching size of G' decreases by one.

The search for an augmenting path runs in $O(n + m)$, which is faster than recomputing the matching from scratch.